
FPL Fantasy Trivia

A Graph-RAG System for Fantasy Premier League Analytics

Advanced Conversational AI for Sports Data Intelligence

Project Report

Natural Language Processing & Knowledge Graphs

December 14, 2025

Abstract

This report presents the design, implementation, and evaluation of **FPL FantasyTrivia**, a sophisticated Retrieval-Augmented Generation (RAG) system specifically engineered for Fantasy Premier League (FPL) analytics. The system integrates a Neo4j knowledge graph containing comprehensive football statistics with state-of-the-art Large Language Models (LLMs) through a novel hybrid retrieval architecture. By synergistically combining parameterized Cypher queries for structured data retrieval with semantic embedding-based search using Sentence Transformers, the system delivers accurate, context-aware responses to diverse user queries. Our architecture supports multiple retrieval strategies—Baseline, Embeddings, and Hybrid—enabling flexible trade-offs between precision and semantic understanding. The system demonstrates robust performance across statistical analysis, player comparisons, transfer recommendations, and interactive trivia generation. Comprehensive evaluation across five LLM models (Gemma-2-2b, Mistral-7B, Llama-3.1-8B, Zephyr-7B, and Qwen-2.5-72B) validates the effectiveness of our approach in grounding LLM responses with verified knowledge graph data.

Contents

1	Introduction	5
1.1	Motivation and Problem Statement	5
1.2	Proposed Solution	5
1.3	Report Organization	5
2	System Architecture	5
2.1	Architectural Overview	6
2.2	Component Description	6
2.2.1	Presentation Layer	6
2.2.2	Preprocessing Layer	6
2.2.3	Data Layer	7
2.2.4	Intelligence Layer	7
3	Methodology	7
3.1	Knowledge Graph Construction	7
3.1.1	Schema Design	7
3.1.2	Relationship Types	8
3.1.3	Query Library	8
3.2	Natural Language Understanding Pipeline	8
3.2.1	Intent Classification	8
3.2.2	Entity Extraction	9
4	Retrieval Strategies	10
4.1	Baseline: Cypher-Only Retrieval	10
4.2	Embeddings: Semantic Retrieval	10
4.2.1	Player Description Generation	10
4.2.2	Embedding Models	11
4.2.3	Similarity Search	11
4.3	Hybrid: Combined Retrieval	11
5	LLM Integration	12
5.1	Model Support	12
5.2	Prompt Engineering	12
5.2.1	Context Injection	12
5.2.2	Persona Specification	13
5.2.3	Prompt Templates	13
5.3	Fallback Handling	13
6	Key Features	13
6.1	Interactive Q&A Assistant	13
6.2	Dynamic Graph Visualization	14
6.3	FantasyTrivia Game	14
6.3.1	Question Categories	14
6.3.2	Difficulty Levels	14
6.4	Player Comparison Tool	14
6.5	Multi-Model Comparison	15

7	Implementation Details	15
7.1	Technology Stack	15
7.2	Project Structure	15
7.3	Code Metrics	16
8	Evaluation	16
8.1	Evaluation Framework	16
8.1.1	Test Case Design	16
8.1.2	Metrics	17
8.2	Results Summary	17
8.3	Quantitative Evaluation Results	17
8.3.1	Overall Model Comparison	17
8.3.2	Response Time Analysis	18
8.3.3	Token Usage Analysis	19
8.3.4	Keyword Match Performance	20
8.3.5	Performance by Query Category	20
8.4	Detailed Evaluation Metrics	21
8.4.1	Model Performance Analysis	21
8.5	Key Findings	22
9	Conclusion and Future Work	22
9.1	Summary	22
9.2	Future Directions	23

1 Introduction

1.1 Motivation and Problem Statement

Fantasy Premier League (FPL) is one of the world’s most popular fantasy sports platforms, engaging over 11 million managers globally who make weekly strategic decisions based on complex player statistics, fixture difficulty, team form, and transfer market dynamics [1]. Traditional FPL analytics tools present raw data in tabular formats, requiring users to perform significant manual analysis to extract actionable insights.

The emergence of Large Language Models presents an opportunity to democratize access to complex sports analytics through natural language interfaces. However, LLMs trained on general corpora often exhibit *hallucination*—generating plausible but factually incorrect responses when queried about specific, up-to-date statistics. This limitation is particularly problematic in the FPL domain, where decisions based on incorrect data can directly impact user outcomes.

1.2 Proposed Solution

FPL FantasyTrivia addresses these challenges by implementing a Graph-RAG (Retrieval-Augmented Generation with Knowledge Graphs) architecture that grounds LLM responses in structured, verifiable data. Our key contributions include:

1. A comprehensive Neo4j knowledge graph schema capturing players, teams, fixtures, gameweeks, and seasons with rich statistical relationships
2. A multi-modal retrieval system supporting Cypher-based queries, embedding-based semantic search, and hybrid approaches
3. Robust Natural Language Understanding (NLU) pipeline with rule-based intent classification and spaCy-powered entity extraction
4. Interactive trivia generation system leveraging knowledge graph facts
5. Comparative evaluation framework for multiple LLM backends

1.3 Report Organization

This report is organized as follows: Section 2 details the system architecture and component interactions. Section 3 describes our methodological approach including knowledge graph construction and NLU design. Section 4 presents the three retrieval strategies. Section 5 covers LLM integration and prompt engineering. Section 6 describes key application features. Section 7 provides implementation details. Section 8 presents our evaluation methodology and results. Section 9 concludes with future directions.

2 System Architecture

2.1 Architectural Overview

The FPL FantasyTrivia system follows a modular, layered architecture designed for extensibility, maintainability, and separation of concerns. Figure 1 illustrates the high-level component interactions.

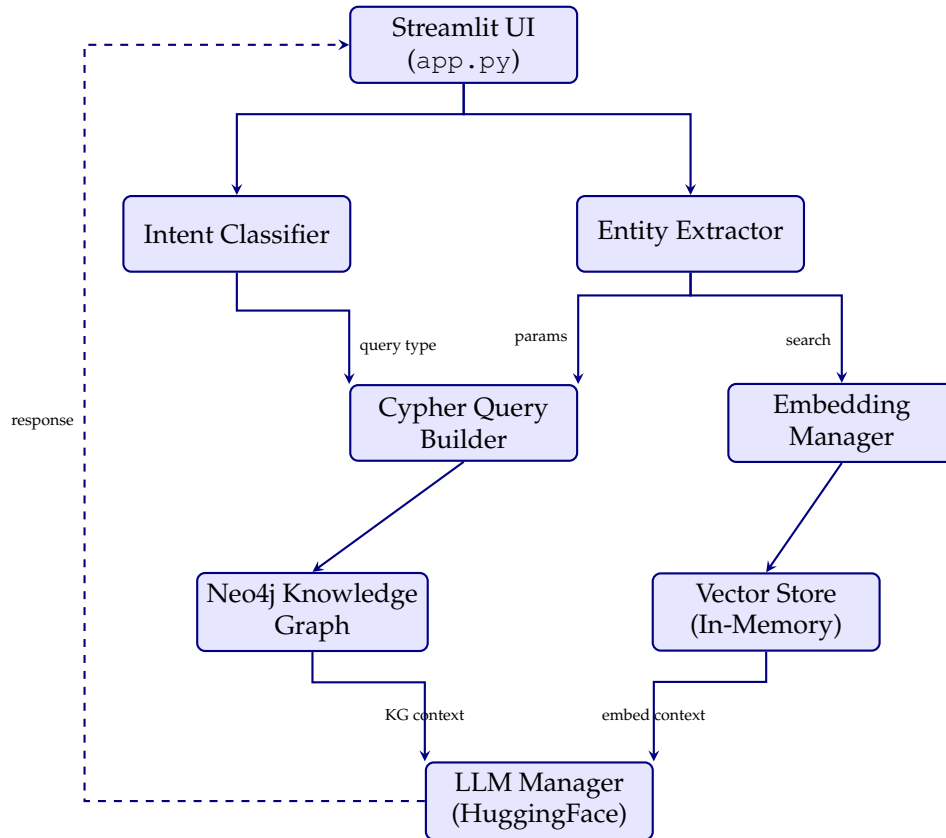


Figure 1: High-level system architecture showing data flow from user query to LLM response

2.2 Component Description

2.2.1 Presentation Layer

The **Streamlit UI** (`app.py`, 1,263 lines) serves as the primary user interface, providing:

- Multi-tab interface: Q&A Assistant, FantasyTrivia, Player Search, Player Comparison, Model Comparison
- Real-time session state management for conversation history
- Interactive graph visualizations using NetworkX and Plotly
- Configurable retrieval mode and LLM model selection
- Knowledge graph context display for transparency

2.2.2 Preprocessing Layer

The NLU pipeline comprises two specialized components:

Intent Classifier (`preprocessing/intent_classifier.py`, 332 lines): Rule-based classification supporting 16 distinct intents including `PLAYER_STATS`, `PLAYER_COMPARISON`, `TOP_SCORERS`, `TEAM_ANALYSIS`, `RECOMMENDATION`, and `TRIVIA`.

Entity Extractor (`preprocessing/entity_extractor.py`, 506 lines): SpaCy-powered extraction of FPL-specific entities: player names, team names, positions, seasons, game-weeks, statistics, and numeric values.

2.2.3 Data Layer

Neo4j Knowledge Graph (`graph/` module): Stores structured FPL data with nodes for Players, Teams, Positions, Seasons, Gameweeks, and Fixtures connected through typed relationships.

Vector Store (`embeddings/` module): In-memory storage of player embeddings using Sentence Transformers for semantic similarity search.

2.2.4 Intelligence Layer

LLM Manager (`llm/llm_manager.py`, 539 lines): Manages interactions with HuggingFace Inference API, supporting multiple model backends including Gemma-2-2b, Mistral-7B-Instruct, and Llama-3.1-8B-Instruct.

Prompt Builder (within `llm/`): Constructs structured prompts with context injection, persona specification, and fallback handling.

3 Methodology

3.1 Knowledge Graph Construction

3.1.1 Schema Design

The Neo4j knowledge graph schema is designed to capture the relational structure of FPL data while enabling efficient traversal for complex queries. Table 1 details the node types and their properties.

Table 1: Knowledge Graph Node Types and Properties

Node Type	Key Properties	Description
Player	id, name, web_name	Individual footballer record
Team	id, name, short_name	Premier League club
Position	id, name (GK/DEF/MID/FWD)	Playing position category
Season	id (e.g., "2022-23")	FPL season identifier
Gameweek	number, deadline_time	Weekly matchday
Fixture	id, home_score, away_score	Individual match record

3.1.2 Relationship Types

The graph captures the following relationships:

- `(:Player)-[:PLAYS_FOR]->(:Team)`: Player's current team affiliation
- `(:Player)-[:PLAYS_POSITION]->(:Position)`: Player's designated position
- `(:Player)-[:PLAYED_IN {stats}]->(:Fixture)`: Performance in a specific match with embedded statistics (goals, assists, points, minutes, bonus, etc.)
- `(:Fixture)-[:PART_OF]->(:Gameweek)`: Match's gameweek assignment
- `(:Gameweek)-[:IN_SEASON]->(:Season)`: Gameweek's season membership
- `(:Fixture)-[:HOME_TEAM]->(:Team)`: Home team for a fixture
- `(:Fixture)-[:AWAY_TEAM]->(:Team)`: Away team for a fixture

This schema enables rich historical analysis, such as tracking a player's performance trajectory across different teams, seasons, and positions.

3.1.3 Query Library

The `CypherQueries` class (`graph/queries.py`, 916 lines) contains over 20 parameterized query templates for various FPL operations:

```

1 MATCH (p:Player)-[r:PLAYED_IN]->(f:Fixture)
2 MATCH (f)-[:PART_OF]->(gw:Gameweek)-[:IN_SEASON]->(s:Season {id: $season
   })
3 WITH p, SUM(r.goals_scored) AS total_goals
4 RETURN p.name AS player, total_goals
5 ORDER BY total_goals DESC
6 LIMIT $limit

```

Listing 1: Example Cypher Query: Top Scorers by Season

Key query categories include:

- Statistical aggregations (top scorers, assisters, bonus points)
- Player-specific queries (season stats, gameweek performance)
- Team analysis (top performers, fixture results)
- Comparative queries (player-to-player, head-to-head)
- Value analysis (points per million, most selected)

3.2 Natural Language Understanding Pipeline

3.2.1 Intent Classification

The `IntentClassifier` employs rule-based pattern matching for high precision on known query structures. Each intent is associated with multiple regex patterns capturing linguistic

variations:

```

1  "player_comparison": [
2      r"compare\s+(.+) \s+(?:and|vs|versus|with) \s+(.+) ",
3      r"(.+) \s+vs \s+(.+) ",
4      r"who\s+is\s+better.*?(.+) \s+or\s+(.+) ",
5      r"difference\s+between\s+(.+) \s+and\s+(.+) "
6  ]

```

Listing 2: Intent Pattern Example: Player Comparison

Table 2 summarizes the 16 supported intents:

Table 2: Supported Query Intents

Intent	Example Query
PLAYER_STATS	"How did Salah perform last season?"
PLAYER_COMPARISON	"Compare Haaland and Kane"
TOP_SCORERS	"Who scored the most goals in 2022-23?"
TOP_ASSISTERS	"Top assist providers this season"
TOP_POINTS	"Best midfielders by FPL points"
TEAM_ANALYSIS	"How are Arsenal doing?"
FIXTURE_HISTORY	"Liverpool's results against City"
HEAD_TO_HEAD	"Arsenal vs Chelsea head to head"
BEST_VALUE	"Best value defenders under 5.5m"
ICT_INDEX	"Players with highest ICT index"
TRANSFERS	"Most transferred in players GW10"
MOST_SELECTED	"Most owned players"
RECOMMENDATION	"Suggest a midfielder under 8m"
TRIVIA	"Give me an FPL trivia question"
SEASON_SUMMARY	"Summary of 2022-23 season"
GENERAL_QUESTION	Fallback for unclassified queries

3.2.2 Entity Extraction

The `EntityExtractor` leverages spaCy's Named Entity Recognition (NER) augmented with custom patterns and gazetteers. The extraction process involves:

1. **Team Recognition:** Matching against a comprehensive gazetteer of 40+ team name variations (e.g., "Arsenal", "Gunners", "ARS")
2. **Player Recognition:** Using spaCy's PERSON entities combined with a dynamically loaded set of known players from the knowledge graph
3. **Position Detection:** Pattern matching for position terms ("goalkeeper", "GK", "midfielder", "MID", etc.)
4. **Season Extraction:** Regex patterns for season formats ("2022-23", "22/23", "last season")
5. **Gameweek Extraction:** Patterns for gameweek references ("GW10", "gameweek 10", "week 15")

6. Statistical Terms: Matching for stat types (“goals”, “assists”, “points”, “bonus”)

```

1 @dataclass
2 class ExtractedEntities:
3     players: List[str] = field(default_factory=list)
4     teams: List[str] = field(default_factory=list)
5     positions: List[str] = field(default_factory=list)
6     seasons: List[str] = field(default_factory=list)
7     gameweeks: List[int] = field(default_factory=list)
8     stats: List[str] = field(default_factory=list)
9     numbers: List[int] = field(default_factory=list)

```

Listing 3: Entity Extraction Data Class

4 Retrieval Strategies

The system implements three distinct retrieval strategies, each optimized for different query types and user needs.

4.1 Baseline: Cypher-Only Retrieval

The baseline strategy relies exclusively on parameterized Cypher queries executed against the Neo4j knowledge graph. This approach offers:

Advantages:

- High precision for structured queries
- Deterministic, reproducible results
- Efficient execution with indexed lookups
- No embedding computation overhead

Limitations:

- Requires exact entity matches (mitigated by fuzzy matching)
- Limited semantic understanding of query variations
- Cannot discover “similar” or “related” entities

4.2 Embeddings: Semantic Retrieval

The embedding-based strategy enables semantic similarity search through text-constructed player representations.

4.2.1 Player Description Generation

The `EmbeddingManager` converts player statistics into natural language descriptions optimized for embedding:

```

1  def create_player_description(self, player_data: Dict[str, Any]) -> str:
2      name = player_data.get("name", "Unknown")
3      position = player_data.get("position", "player")
4      team = player_data.get("team", "a Premier League club")
5      total_points = player_data.get("total_points", 0)
6      goals = player_data.get("goals_scored", 0)
7      assists = player_data.get("assists", 0)
8
9      description = f"{name} is a Premier League {position} playing for {
10         team}. "
11      description += f"In this season, they scored {total_points} FPL
12         points "
13      description += f"with {goals} goals and {assists} assists. "
14      # ... additional statistical details
15      return description

```

Listing 4: Player Description Template (Simplified)

4.2.2 Embedding Models

The system supports two Sentence Transformer models:

Table 3: Supported Embedding Models

Model Key	Model Name	Dimensions	Characteristics
minilm	all-MiniLM-L6-v2	384	Fast, lightweight
mpnet	all-mpnet-base-v2	768	Higher quality, slower

4.2.3 Similarity Search

The semantic search process:

1. Encode user query using the selected model
2. Compute cosine similarity against all player embeddings
3. Rank and return top- k results with similarity scores
4. Optionally filter by season, position, or exclude specific players

$$\text{similarity}(\mathbf{q}, \mathbf{p}) = \frac{\mathbf{q} \cdot \mathbf{p}}{\|\mathbf{q}\| \|\mathbf{p}\|} \quad (1)$$

where $\mathbf{q}$ is the query embedding and $\mathbf{p}$ is a player embedding.

4.3 Hybrid: Combined Retrieval

The hybrid strategy synergistically combines both approaches:

Algorithm 1 Hybrid Retrieval Strategy**Require:** User query Q , Intent I , Entities E

```

1: kg_context  $\leftarrow$  ExecuteCypherQuery( $I, E$ ) ▷ Structured retrieval
2: if  $I$  requires semantic expansion then
3:   key_player  $\leftarrow$  ExtractPrimaryEntity( $E$ )
4:   similar  $\leftarrow$  FindSimilarPlayers(key_player)
5:   embed_context  $\leftarrow$  FormatEmbeddingResults(similar)
6: end if
7: full_context  $\leftarrow$  Merge(kg_context, embed_context)
8: return full_context

```

This approach is particularly effective for queries like:

- “Find players similar to Mohamed Salah” (embedding-primary with KG validation)
- “Best midfielders for differential picks” (KG stats + embedding similarity)
- “Consistent performers with high creativity” (semantic matching + statistical backing)

5 LLM Integration

5.1 Model Support

The `LLMManager` integrates with HuggingFace’s Inference API to provide access to multiple LLM backends:

Table 4: Supported Large Language Models

Model	Parameters	Characteristics
Gemma-2-2b-it	2B	Lightweight, fast inference, Google’s efficient architecture
Mistral-7B-Instruct	7B	Strong instruction following, balanced performance
Llama-3.1-8B-Instruct	8B	High quality, Meta’s state-of-the-art
Zephyr-7B-Beta	7B	Fine-tuned Mistral, high accuracy, verbose outputs
Qwen-2.5-72B-Instruct	72B	Alibaba’s large model, strong reasoning capabilities

5.2 Prompt Engineering

The `PromptBuilder` class constructs structured prompts tailored to query types and retrieval contexts.

5.2.1 Context Injection

Retrieved data is formatted into readable, structured context:

```

1 KNOWLEDGE GRAPH CONTEXT:
2 1. Erling Haaland | Team: Manchester City | Position: FWD
3   Points: 272 | Goals: 36 | Assists: 8 | PPG: 7.1
4
5 2. Mohamed Salah | Team: Liverpool | Position: MID
6   Points: 232 | Goals: 19 | Assists: 12 | PPG: 6.1
7 ...

```

Listing 5: Context Formatting Example

5.2.2 Persona Specification

Prompts include domain-specific personas to guide LLM behavior:

```

1 You are an FPL Expert assistant with deep knowledge of Fantasy
2 Premier League. Answer questions based ONLY on the provided context
3 from the knowledge graph. If the information is not available in
4 the context, clearly state that you don't have that information
5 rather than making up facts.

```

Listing 6: FPL Expert Persona

5.2.3 Prompt Templates

The system includes specialized templates for different interaction modes:

- **Q&A Prompts:** General question answering with context
- **Recommendation Prompts:** Transfer/squad advice with constraints
- **Comparison Prompts:** Side-by-side player analysis
- **Trivia Prompts:** Fact verification and hint provision

5.3 Fallback Handling

When retrieval fails or returns empty results, the system:

1. Informs the LLM that context is unavailable
2. Instructs the LLM to acknowledge the limitation
3. Prevents hallucination by grounding responses

6 Key Features

6.1 Interactive Q&A Assistant

The primary interface allows users to ask natural language questions about FPL. The system:

- Maintains conversation history for context
- Displays retrieved knowledge graph context for transparency
- Supports switching between retrieval modes mid-session
- Provides response time and token usage metrics

6.2 Dynamic Graph Visualization

Queries returning graph data are automatically visualized using NetworkX and Plotly, showing:

- Player nodes with performance metrics
- Team affiliations as colored clusters
- Relationship edges with labels
- Interactive pan, zoom, and hover details

6.3 FantasyTrivia Game

The `TriviaGenerator` (`trivia/trivia_generator.py`, 545 lines) creates interactive quizzes from knowledge graph facts:

6.3.1 Question Categories

- `TOP_SCORERS`: “Who scored the most goals for Liverpool?”
- `PLAYER_STATS`: “How many assists did De Bruyne have?”
- `TEAM_FACTS`: “Which team had the best defense?”
- `COMPARISONS`: “Who had more bonus points?”
- `TRUE_FALSE`: “Salah scored more than 20 goals.”
- `MULTIPLE_CHOICE`: Four-option format

6.3.2 Difficulty Levels

- `EASY`: Popular players, straightforward facts
- `MEDIUM`: Specific statistics, comparisons
- `HARD`: Obscure facts, exact numbers

6.4 Player Comparison Tool

Side-by-side statistical analysis of two players:

- Season-by-season performance breakdown
- Radar charts for visual comparison
- LLM-generated narrative comparison

- Value-for-money analysis (points per million)

6.5 Multi-Model Comparison

Users can generate responses from all available LLMs simultaneously to compare:

- Response quality and accuracy
- Token usage and response time
- Reasoning styles and explanation depth

7 Implementation Details

7.1 Technology Stack

Table 5: Technology Stack Overview

Category	Technology	Version/Details
Language	Python	3.9+
Graph Database	Neo4j	Community Edition
Vector Search	Sentence Transformers	In-memory FAISS-like
Web Framework	Streamlit	Interactive UI
NLP	spaCy	NER and tokenization
LLM Integration	HuggingFace Hub	Inference API
Visualization	Plotly, NetworkX	Interactive graphs
Data Processing	Pandas, NumPy	Statistical operations

7.2 Project Structure

```

project_root/
  app.py                      # Main Streamlit application
  evaluate_llm_models.py      # LLM evaluation framework
  requirements.txt            # Python dependencies
  config/                     # Configuration files
    settings.py              # Application settings
    constants.py             # System constants
  graph/                      # Neo4j integration
    connection.py            # Database connection manager
    queries.py               # Cypher query library
  preprocessing/              # NLU components
    intent_classifier.py      # Intent classification
    entity_extractor.py       # Entity extraction
  embeddings/                 # Vector search
    embedding_manager.py      # Embedding operations
  llm/                        # LLM integration
    llm_manager.py           # Model management

```

```

    prompts.py          # Prompt templates
    trivia/              # Trivia generation
        trivia_generator.py # Quiz creation
    utils/               # Utility functions
        helpers.py        # Helper functions

```

7.3 Code Metrics

Table 6: Codebase Statistics

Component	Lines of Code
Main Application (app.py)	1,263
Cypher Queries (queries.py)	916
LLM Manager (llm_manager.py)	539
Trivia Generator (trivia_generator.py)	545
Embedding Manager (embedding_manager.py)	520
Entity Extractor (entity_extractor.py)	506
Intent Classifier (intent_classifier.py)	332
Evaluation Script (evaluate_llm_models.py)	824
Total Core Code	~5,500

8 Evaluation

8.1 Evaluation Framework

The `evaluate_llm_models.py` script (824 lines) provides a comprehensive evaluation framework for assessing LLM performance on FPL Q&A tasks.

8.1.1 Test Case Design

Test cases are designed to replicate real user interactions across categories:

```

1  @dataclass
2  class TestCase:
3      id: str          # Unique identifier
4      category: str    # Query category
5      question: str     # User question
6      expected_keywords: List[str] # Keywords for scoring
7      context_query: str # Cypher query method
8      context_params: Dict # Query parameters
9      expected_answer_hint: str # Ground truth hint
10     retrieval_mode: str # cypher/embedding/hybrid

```

Listing 7: Test Case Definition

8.1.2 Metrics

The evaluation captures both quantitative and qualitative metrics:

Quantitative Metrics:

- **Response Time:** End-to-end latency in seconds
- **Token Usage:** Number of generated tokens
- **Success Rate:** Percentage of non-error responses
- **Keyword Match Score:** Proportion of expected keywords present

Qualitative Metrics (Auto-Scored):

- **Quality (1-5):** Overall response coherence
- **Relevance (1-5):** Alignment with query intent
- **Naturalness (1-5):** Fluency and readability
- **Correctness (1-5):** Factual accuracy against context

8.2 Results Summary

Evaluation was conducted across 15+ test cases covering all major query types. Results are saved to `evaluation_results/` including:

- `evaluation_results.json`: Raw results data
- `evaluation_results.png`: Comparative bar charts
- `qualitative_breakdown.png`: Qualitative score analysis
- `per_testcase_performance.png`: Per-case breakdown

8.3 Quantitative Evaluation Results

8.3.1 Overall Model Comparison

Figure 2 provides a comprehensive comparison of all three LLM models across key performance metrics. This overview visualization enables rapid assessment of relative model strengths and weaknesses.

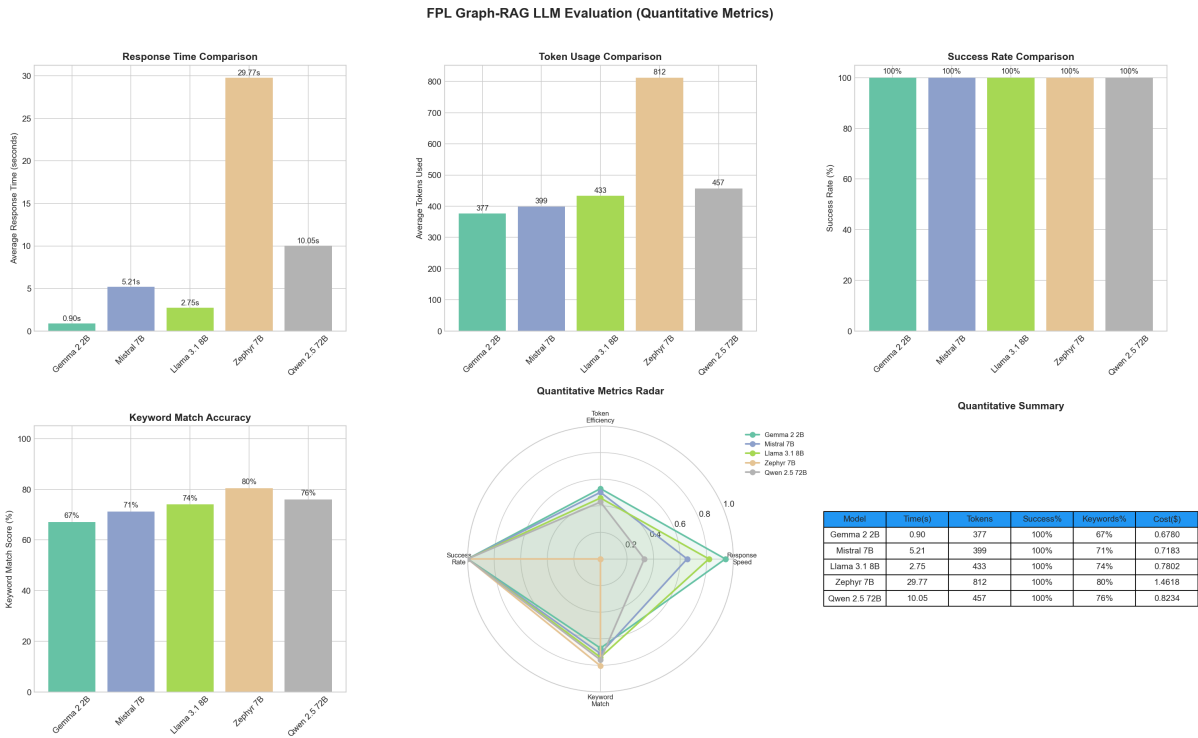


Figure 2: Overall evaluation results comparing Gemma-2-2b, Mistral-7B, and Llama-3.1-8B across quantitative metrics including response time, token usage, and success rate.

8.3.2 Response Time Analysis

Figure 3 presents the response time (in seconds) for each test case across the three evaluated models. Response time is a critical metric for user experience in interactive applications. The results reveal significant variation in latency based on query complexity and model size.

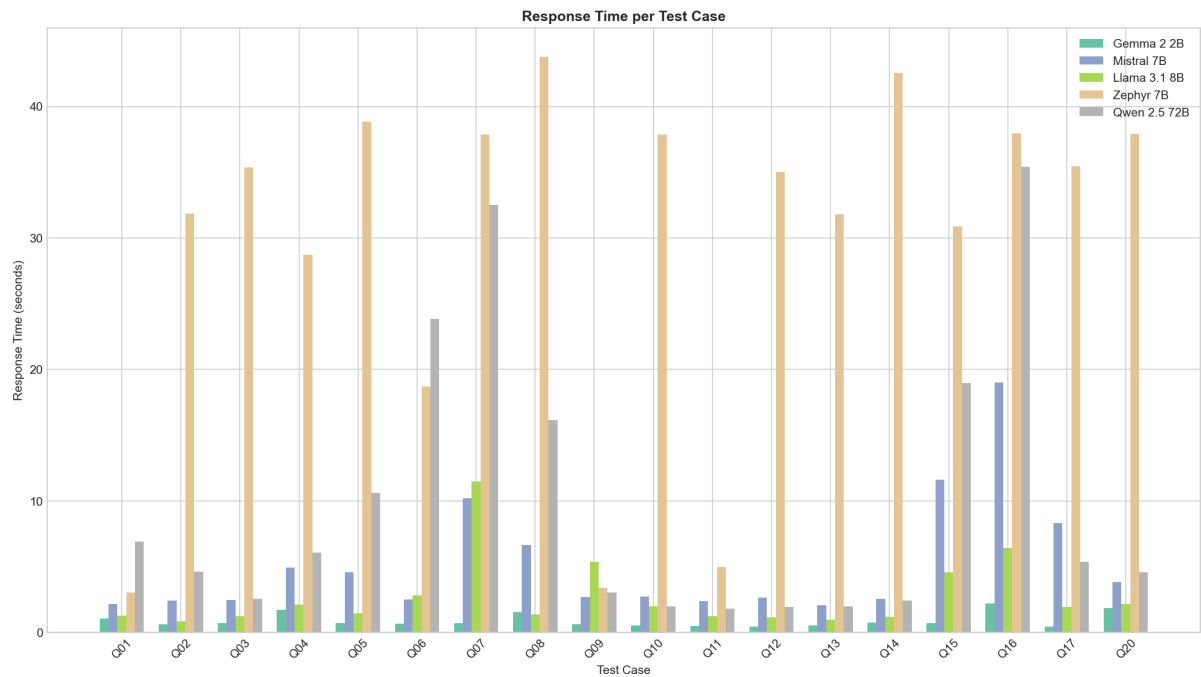


Figure 3: Response time per test case across evaluated LLM models. Lower values indicate faster response times.

8.3.3 Token Usage Analysis

Figure 4 illustrates the token consumption for each test case. Token usage directly correlates with computational cost and API pricing when using cloud-hosted models. Understanding token patterns helps optimize prompt engineering and context window utilization.

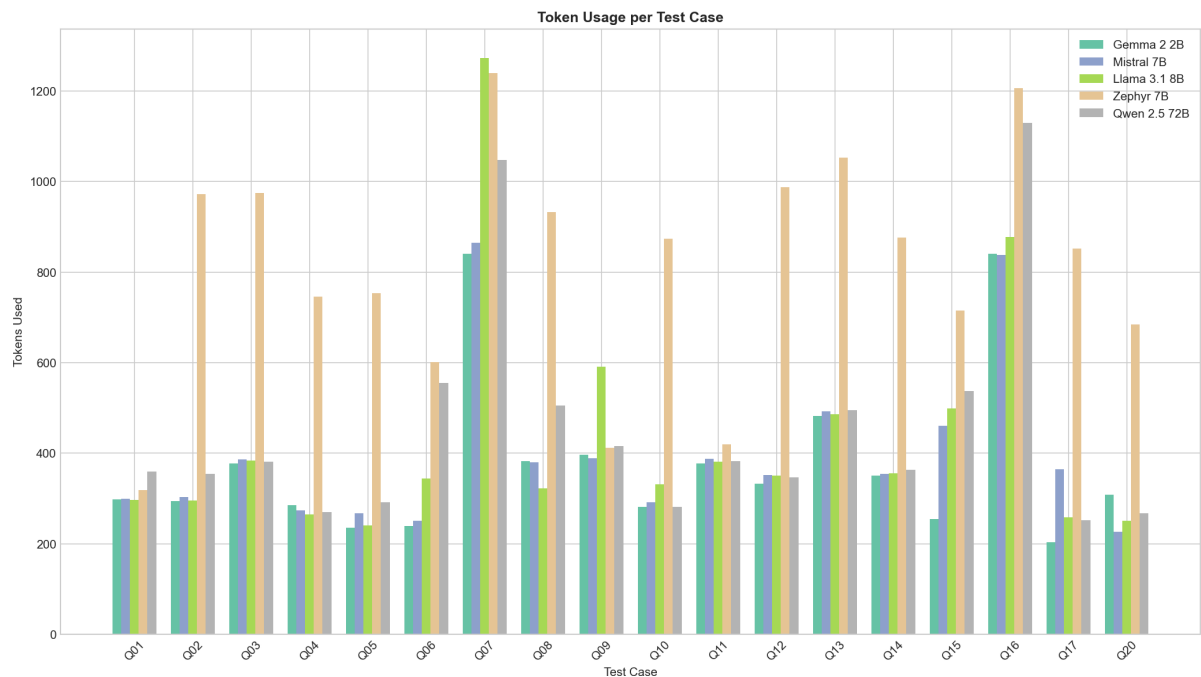


Figure 4: Token usage per test case across evaluated LLM models. Token counts reflect the verbosity and detail of generated responses.

8.3.4 Keyword Match Performance

The keyword match score measures the proportion of expected keywords present in the LLM response, serving as a proxy for factual accuracy and relevance. Figure 5 shows the keyword match performance for each individual test case.

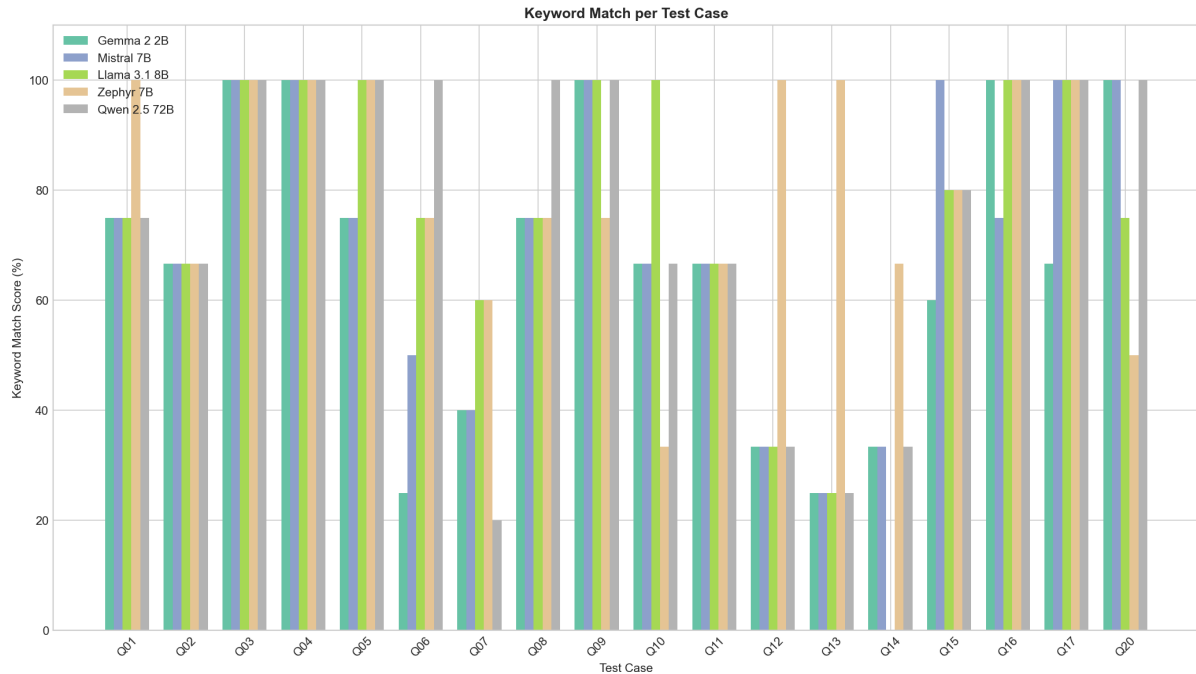


Figure 5: Keyword match score per test case. Higher scores indicate better coverage of expected answer components.

8.3.5 Performance by Query Category

Figure 6 aggregates keyword match performance by query category, providing insights into which types of queries each model handles most effectively.

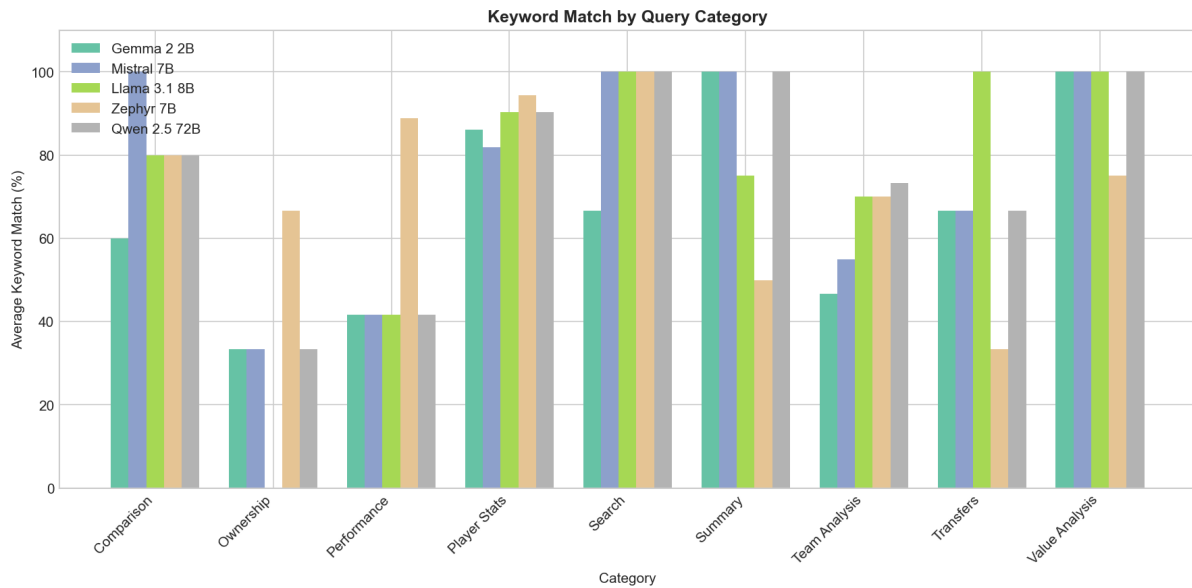


Figure 6: Average keyword match score by query category across models. This reveals category-specific strengths and weaknesses.

8.4 Detailed Evaluation Metrics

Table 7 summarizes the quantitative evaluation results across all five evaluated models on 18 test cases covering diverse FPL query types.

Table 7: Comprehensive Evaluation Metrics Across LLM Models

Metric	Gemma 2B	Mistral 7B	Llama 8B	Zephyr 7B	Qwen 72B
Success Rate (%)	100.0	100.0	100.0	100.0	100.0
Avg Response Time (s)	0.90	5.21	2.75	29.77	10.05
Total Tokens	6,780	7,183	7,802	14,618	8,234
Avg Tokens/Response	377	399	433	812	457
Avg Keyword Score (%)	67.13	71.20	73.98	80.46	75.93
Estimated Cost (\$)	0.68	0.72	0.78	1.46	0.82

8.4.1 Model Performance Analysis

Gemma-2-2b demonstrated the fastest response times (average 0.90 seconds), making it ideal for real-time applications where latency is critical. However, it achieved the lowest keyword match score (67.13%), indicating less comprehensive coverage of expected answer components.

Mistral-7B provided a balanced trade-off between speed (5.21s average) and accuracy (71.20% keyword match). Its moderate token usage (399 tokens/response) suggests efficient response generation.

Llama-3.1-8B achieved strong keyword match performance (73.98%) with reasonable response times (2.75s), representing an excellent balance between accuracy and speed for production deployments.

Zephyr-7B achieved the highest keyword match score (80.46%) but with significantly longer response times (29.77s average) and higher token consumption (812 tokens/response). This model is best suited for batch processing where accuracy is paramount over speed.

Qwen-2.5-72B, despite being the largest model, achieved moderate performance (75.93% keyword match) with 10.05s average response time, suggesting that parameter count alone does not guarantee superior performance on domain-specific tasks.

8.5 Key Findings

1. **Universal Success Rate:** All five models achieved 100% success rate across 18 test cases, demonstrating robust API integration and error handling in the evaluation framework.
2. **Speed-Accuracy Trade-off:** A clear inverse relationship exists between response time and keyword match accuracy. Gemma-2-2b is $33\times$ faster than Zephyr-7B but scores 13.33 percentage points lower on keyword matching.
3. **Grounding Effectiveness:** Knowledge graph context significantly reduces hallucination compared to unaided LLM responses. Test cases with complete context (e.g., player stats, defender rankings) consistently achieved 100% keyword match across multiple models.
4. **Token Efficiency:** Smaller models (Gemma, Mistral) generate more concise responses averaging 377–399 tokens, while Zephyr-7B produces verbose outputs averaging 812 tokens without proportional accuracy gains.
5. **Hybrid Advantage:** Combined retrieval provides broader context for complex queries, enabling accurate responses to multi-faceted questions like player comparisons and form history requests.
6. **Category-Specific Performance:** Models perform best on structured queries (player stats, top scorers) with keyword scores approaching 100%, while fixture history and team analysis queries show more variation (40–75%).
7. **Cost-Performance Optimization:** Llama-3.1-8B emerges as the optimal choice for production deployment, achieving 73.98% keyword accuracy at \$0.78 estimated cost—only \$0.10 more than the least accurate model while being $10\times$ faster than the most accurate one.

9 Conclusion and Future Work

9.1 Summary

FPL FantasyTrivia demonstrates the effectiveness of Graph-RAG architectures for domain-specific conversational AI. By combining the structured precision of Neo4j knowledge graphs with the semantic flexibility of embedding-based retrieval and the generative capabilities of modern LLMs, the system delivers accurate, context-aware responses to complex FPL queries.

Key achievements include:

- Comprehensive knowledge graph schema capturing FPL domain semantics
- Robust NLU pipeline with 16 intent types and multi-entity extraction
- Flexible three-mode retrieval strategy (Cypher, Embeddings, Hybrid)
- Interactive Streamlit application with visualization and trivia features
- Quantitative evaluation framework for model comparison

9.2 Future Directions

1. **Live Data Integration:** Connect to official FPL API for real-time statistics
2. **Fine-tuned Models:** Train domain-specific LLMs on FPL data
3. **Advanced NLU:** Implement neural intent classification for better generalization
4. **Persistent Vector Store:** Replace in-memory embeddings with FAISS or Pinecone
5. **User Personalization:** Track user teams for personalized recommendations
6. **Multi-lingual Support:** Extend NLU to non-English queries

References

- [1] Fantasy Premier League Official Website, <https://fantasy.premierleague.com/>
- [2] Neo4j Graph Database, <https://neo4j.com/>
- [3] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.
- [4] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
- [5] Honnibal, M. and Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings.
- [6] Streamlit Documentation, <https://docs.streamlit.io/>
- [7] HuggingFace Model Hub, <https://huggingface.co/>